

Prog 5/6

Introduction

Johan Korten johan.korten@han.nl

Controlling a stepper

- Programming a stepper motor
- The ESE dilemma
- The Prog 5/6 course
- Programming a stepper motor better

Controlling a stepper

- What do we need?
- How do we keep track of what we need?

Controlling a stepper

See code example `stepper_code_example_1a.ino`

- What are the pro's of this approach?
- What are the con's?

Controlling a stepper

- What are the pro's of this approach?
- What are the con's?

Controlling a stepper

- What are the pro's of this approach?

Another approach (MIC2-style ;)

- Directly program all registers

```
#include <avr/io.h>

void setup() {
    // D2 (PD2)=STEP, D3 (PD3)=DIR, D4 (PD4)=ENA as outputs
    DDRD |= (1<<2) | (1<<3) | (1<<4);

    // Enable driver (assumes ENA active-LOW), STEP=LOW, DIR=LOW
    PORTD &= ~(1<<4);
    PORTD &= ~((1<<2) | (1<<3));

    // Timer1: normal mode, prescaler = 8 (16 MHz / 8 = 2 MHz → 0.5 μs/tick)
    TCCR1A = 0;
    TCCR1B = (1<<CS11);
    // No Timer1 interrupts used
    TIMSK1 = 0;
}
```

```

void loop() {
  // ---- Clockwise: DIR HIGH, settle ~20  $\mu$ s (40 ticks) ----
  PORTD |= (1<<3);
  uint16_t t = TCNT1;
  while ((uint16_t)(TCNT1 - t) < 40) { } // ~20  $\mu$ s

  // 1 rev @ 16 $\times$  microstep = 3200 pulses
  for (unsigned int i = 0; i < 3200; i++) {
    PORTD |= (1<<2);      // STEP HIGH
    t = TCNT1; while ((uint16_t)(TCNT1 - t) < 20) { } // ~10  $\mu$ s HIGH
    PORTD &= ~(1<<2);     // STEP LOW
    t = TCNT1; while ((uint16_t)(TCNT1 - t) < 20) { } // ~10  $\mu$ s LOW
  }

  // ~500 ms pause using Timer1: 25 ms (50,000 ticks)  $\times$  20
  for (uint8_t k = 0; k < 20; k++) {
    t = TCNT1; while ((uint16_t)(TCNT1 - t) < 50000) { } // ~25 ms
  }

  // ---- Counterclockwise: DIR LOW, settle ~20  $\mu$ s ----
  ...

```

```

void loop() {
    // ---- Clockwise: DIR HIGH, settle ~20  $\mu$ s (40 ticks) ----
    ...

    // ---- Counterclockwise: DIR LOW, settle ~20  $\mu$ s ----
    PORTD &= ~(1<<3);
    t = TCNT1;
    while ((uint16_t)(TCNT1 - t) < 40) { } // ~20  $\mu$ s

    // 1 rev back
    for (unsigned int i = 0; i < 3200; i++) {
        PORTD |= (1<<2);
        t = TCNT1; while ((uint16_t)(TCNT1 - t) < 20) { }
        PORTD &= ~(1<<2);
        t = TCNT1; while ((uint16_t)(TCNT1 - t) < 20) { }
    }

    // ~500 ms pause again
    for (uint8_t k = 0; k < 20; k++) {
        t = TCNT1; while ((uint16_t)(TCNT1 - t) < 50000) { }
    }
}

```

Controlling a stepper

- What are the pro's of this approach?
- What are the con's?

Controlling a stepper

- What are the pro's of this approach?

The ESE-dilemma

- How do we program prototypes?

The ESE-dilemma

- How do we program prototypes?
- How do we program pacemakers?

The ESE-dilemma

- To summarize the first part of this introduction:
 - No matter what they tell you:
 - Embedded Software Engineering is:
 - The most vital subject of ESE
 - The most difficult subject of ESE
- People WILL die if we make a mess (more on that in MIC6 ;))
- (If you don't actively participate you won't get buck for you money...)

The ESE-dilemma

- We work in GitHub:
- https://github.com/AEAEmbedded/ESE_PROG.git
- GitHub Classroom 2026-2027: link follows on Brightspace
- Send me your GitHub username so I can link it to your account

The ESE-dilemma

- To summarize the first part of this introduction:
 - No matter what they tell you:
 - Embedded Software Engineering is:
 - The most vital subject of ESE
 - The most difficult subject of ESE
 - People WILL die if we make a mess (more on that in MIC6 ;))

The ESE-dilemma

- Why ESE is really engineering:
 - Trade-off between readability and functionality
 - Efficient code might not be as readable
 - Adding abstractions will cost overhead (memory, performance)
 - Even well designed code is a trade-off between memory and processor intensiveness

Let's review `stepper_code_example_1a.ino`

Programming in MIC2 style

Disclaimer: Of course in MIC2 we wanted to show you how microcontrollers work under the hood. Different didactical goal for good reason!

The ESE-dilemma

Readability

- Is readable code always reliable?

The ESE-dilemma

Readability

- Is readable code always reliable?
- Is reliable code always readable?

The ESE-dilemma

Readability

- Is readable code always reliable?
- Is reliable code always readable?
- What is readable code?

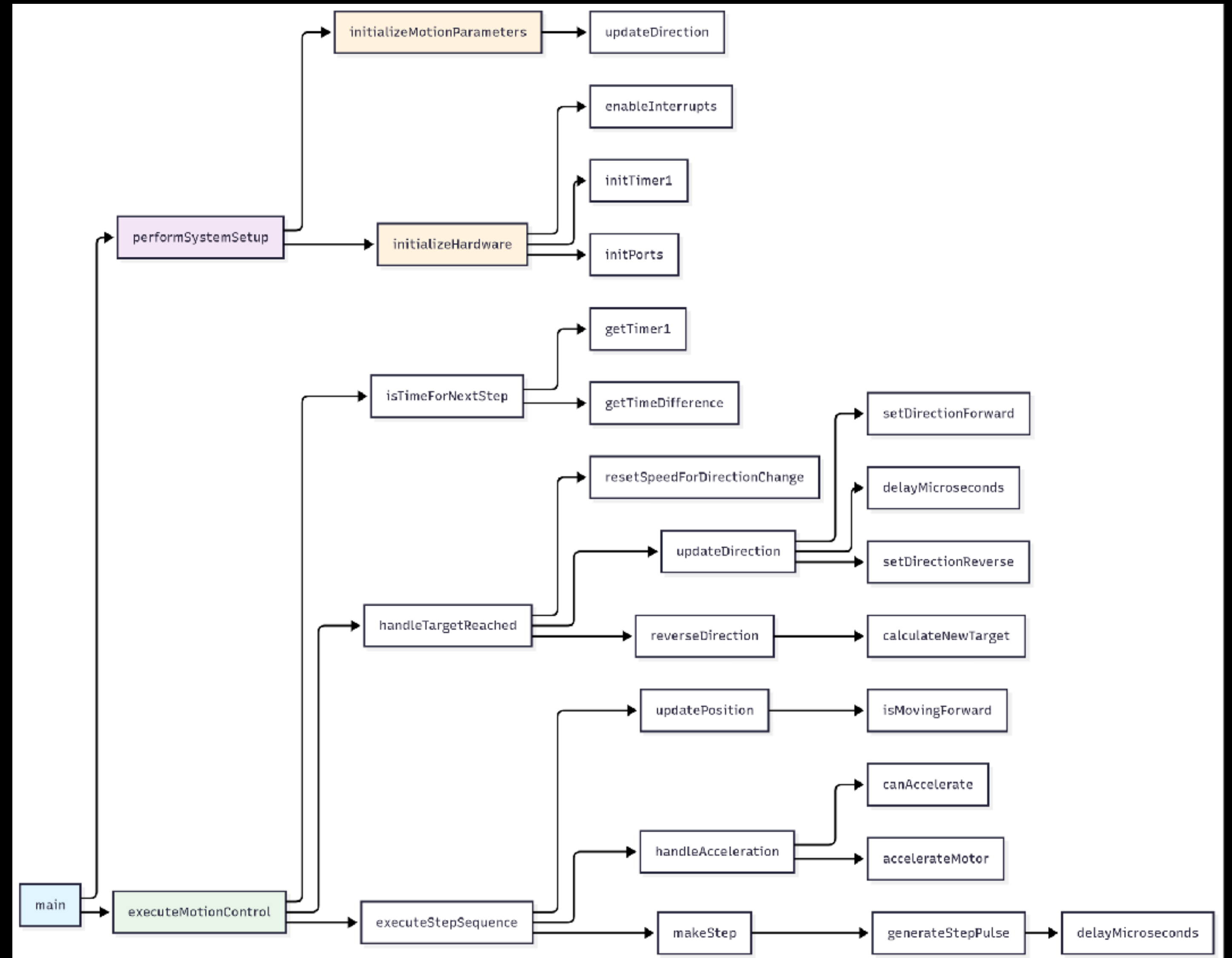
The ESE-dilemma

Readability

- Is readable code always reliable?
 - Is reliable code always readable?
 - What is readable code?
-
- WARNING: Try to avoid hacking (e.g. overruling the compiler, unsafe casting, etc)

Let's review stepper_code_example_1b.ino

Programming in MIC2 style
but now with methods



The ESE-dilemma

Readability

- Is the code structured well?
- Why is the code (not) readable?
- What code smells did you detect?
- Under which circumstance would this code be acceptable (at least partially)

Structured programming

- Define the properties you need
- Define the behavior you need

Structured programming

Properties

- Variables
- Constants (always preferred)
- Static vs non-static

Structured programming

Properties - Variables

- Variables
 - Variables are storage locations that hold data values that can change during program execution. They have:
 - A name (identifier) to reference them
 - A type (in typed languages) that determines what kind of data they can store
 - A value that can be modified (i.e. is mutable)

Structured programming

Properties - Constants

- Constants (Always Preferred)
 - Constants are immutable values that cannot be changed after initialization. They provide:
 - Immutability - prevents accidental modification
 - Performance benefits - compiler optimizations
 - Code clarity - signals intent that value shouldn't change
 - Thread safety - no risk of concurrent modification

Structured programming

Properties

- Static vs Non-Static
 - Static Members
 - Belong to the class itself, not individual instances
 - Shared across all instances of the class
 - Accessed using the class name directly
 - Initialized once when the class is first loaded
 - Non-Static (Instance) Members
 - Belong to specific instances of the class
 - Each object has its own copy
 - Must be accessed through an object instance
 - Initialized each time an object is created

Structured programming

Behavior

- Methods
 - void type:
 - By default: performs some action (in an embedded context this might as well be an action that changes the physical environment i.e. actuator)
 - Warning: try to avoid side effects (i.e. methods that modify properties without explicitly stating this in the method name).
- Methods
 - non-void type:
 - By default: modifies some properties and/or state

Let's review `stepper_code_example_1c.ino`

The ESE-dilemma

Readability

- Is the code structured well?
- Why is the code (not) readable?
- What code smells did you detect?

The ESE-dilemma

Resource management

The screenshot displays the Arduino IDE 2.3.5 interface. The top toolbar shows the upload button (a right-pointing arrow) is disabled, indicating a compilation error. The file explorer on the left shows the project file 'stepper_code_example_1e.ino'. The main editor window contains C++ code for a stepper motor driver, including functions for handling limits and backoff. The bottom panel shows the 'Output' tab with the following error message:

```
Sketch uses 10616 bytes (32%) of program storage space. Maximum is 32256 bytes.  
Global variables use 2332 bytes (113%) of dynamic memory, leaving -284 bytes for local variables. Maximum is 2048 bytes.  
Not enough memory; see https://support.arduino.cc/hc/en-us/articles/360013825179 for tips on reducing your footprint.  
data section exceeds available space in board  
  
Compilation error: data section exceeds available space in board
```

A notification box at the bottom right of the IDE displays the error message: 'Compilation error: data section exceeds available space in board'. A button labeled 'COPY ERROR MESSAGES' is located next to the notification. The status bar at the bottom indicates the current position is 'Ln 823, Col 14' and the board is 'Arduino Uno on /dev/cu.usbmodem111401'.

Course

- Send me your student number + GitHub account name
- I will only use Brightspace for practical information as this course tries to come as close to real development as possible.
- Tips to get the most from this course:
 - Come prepared
 - Practice makes perfect!
 - You can never program too much
 - Vibe-coding: typically avoid, however useful for diagramming, documenting and refactoring, code-reviewing, use with caution

One more thing...

- Check code against code standards (e.g. MISRA)
- Use cppcheck and other static code analyzers
- Carefully check:
 - Your code
 - Your colleagues' (pair programming, code reviews)
 - Your teachers' (including me)
- TIP: Make use of git pull request and issues